

Data Preprocessing & Visualization

Session 1: (Numpy)

Agenda:

1	Introduction to Numpy
2	Numpy Arrays
3	Practical Applications

Introduction to Numpy

Introduction to Numpy

What is Numpy?

Numpy is a fundamental package for scientific computing in Python. It is used for performing numerical operations on large, multi-dimensional arrays and matrices. It also provides a collection of mathematical functions to operate on these arrays.

Numpy is essential for data science, machine learning, and scientific computing because of its efficient array operations and broadcasting capabilities.

Introduction to Numpy

Key Features of Numpy

- **N-dimensional arrays:** Numpy provides support for multi-dimensional arrays and matrices, which are more efficient than Python lists.
- **Broadcasting:** Numpy can perform operations on arrays of different shapes and sizes, automatically handling dimension mismatches.
- **Mathematical functions:** Numpy includes functions for mathematical operations such as linear algebra, statistics, and trigonometry.
- **Integration with other libraries:** Numpy is the foundation for many other scientific libraries in Python, such as Pandas, Scipy, and Scikit-learn.

Numpy Arrays

Creating Numpy Arrays

Creating a 1D Numpy Array from a List

```
import numpy as np

arr1 = np.array([1, 2, 3, 4, 5])
print(arr1) # Output: [1 2 3 4 5]
```

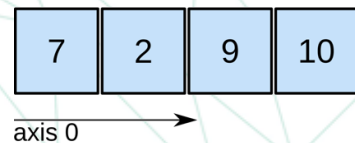
[1 2 3 4 5]

This creates a one-dimensional Numpy array `arr1` from a Python list. Numpy arrays are more efficient for numerical operations than Python lists.

Shape of NumPy Arrays

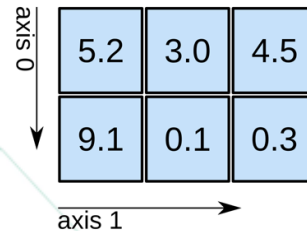
The shape of a NumPy array is represented as a tuple, indicating the number of elements present in each dimension. It provides information about the size of the array across its various dimensions.

1D array



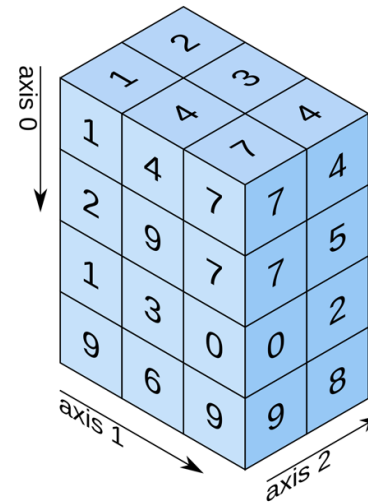
shape: (4,)

2D array



shape: (2, 3)

3D array



shape: (4, 3, 2)

Creating Numpy Arrays

Creating a 2D Numpy Array from a List of Lists

```
arr2 = np.array([[1, 2, 3], [4, 5, 6]])  
print(arr2) # Output: [[1 2 3]  
#                [4 5 6]]
```

```
[[1 2 3]  
 [4 5 6]]
```

This creates a two-dimensional Numpy array `arr2` from a list of lists. Numpy arrays can handle multi-dimensional data easily.

Creating Numpy Arrays

Creating an Array of Zeros

```
zeros = np.zeros((2, 3))  
print(zeros) # Output: [[0. 0. 0.]  
#                [0. 0. 0.]
```

```
[[0. 0. 0.]  
 [0. 0. 0.]
```

This creates a 2x3 Numpy array filled with zeros. It is useful for initializing arrays with default values.

Creating Numpy Arrays

Creating an Array of Ones

```
[21] ones = np.ones((3, 2))  
      print(ones) # Output: [[1. 1.]  
                          #      [1. 1.]  
                          #      [1. 1.]
```

```
⇒ [[1. 1.]  
    [1. 1.]  
    [1. 1.]
```

This creates a 3x2 Numpy array filled with ones. It's useful for creating arrays where all elements start with a value of one.

Creating Numpy Arrays

Creating an Array with a Specific Range

```
▶ range_array = np.arange(10)  
print(range_array) # Output: [0 1 2 3 4 5 6 7 8 9]
```

```
⇒ [0 1 2 3 4 5 6 7 8 9]
```

This creates a one-dimensional array of integers from 0 to 9 using `np.arange()`. It's similar to Python's built-in `range()` but returns a Numpy array.

Creating Numpy Arrays

Creating an Array with a Range of Values and Specific Step Size

```
[23] range_step = np.arange(0, 10, 2)  
     print(range_step) # Output: [0 2 4 6 8]
```

⇒ [0 2 4 6 8]

This creates an array with values starting from 0 up to (but not including) 10, with a step size of 2. Useful for generating sequences with a specific interval.

Creating Numpy Arrays

Creating an Array of Evenly Spaced Values Between Two Numbers

```
▶ linspace_array = np.linspace(0, 1, 5)  
print(linspace_array) # Output: [0.  0.25 0.5  0.75 1.  ]
```

```
↔ [0.  0.25 0.5  0.75 1.  ]
```

This creates an array of 5 evenly spaced values between 0 and 1. `np.linspace()` is often used for generating a sequence of numbers over a specified interval.

Creating Numpy Arrays

Creating an Array with Random Values Between 0 and 1

```
▶ random_array = np.random.rand(3, 3)  
print(random_array)
```

```
↔ [[0.86442628 0.74889114 0.76632541]  
   [0.35276105 0.24361501 0.52784889]  
   [0.61494782 0.07521612 0.99683728]]
```

This creates a 3x3 array with random floating-point numbers between 0 and 1. Useful for simulations and generating random datasets.

Creating Numpy Arrays

Creating an Array with Random Integers

```
random_integers = np.random.randint(1, 10, (2, 2))  
print(random_integers)
```

```
[[3 4]  
 [8 6]]
```

This creates a 2x2 array with random integers between 1 and 9. Ideal for testing and generating sample data.

Creating Numpy Arrays

Checking the Shape of an Array

```
▶ arr = np.array([[1, 2, 3], [4, 5, 6]])  
print(arr.shape) # Output: (2, 3)
```

⇒ (2, 3)

This returns the shape of the array, showing it has 2 rows and 3 columns. shape is useful for understanding the structure of multi-dimensional arrays.

Creating Numpy Arrays

Accessing Data Type of Elements

```
print(arr.dtype) # Output: int64
```

```
int64
```

This checks the data type of elements in the array, showing that they are 64-bit integers. Knowing the data type helps optimize storage and computation.

Creating Numpy Arrays

Indexing a 1D Array

```
▶ arr = np.array([10, 20, 30, 40, 50])  
print(arr[0]) # Output: 10  
print(arr[-1]) # Output: 50
```

```
⇒ 10  
50
```

This accesses elements in the array by their index. Positive indices start from the beginning, while negative indices start from the end.

Creating Numpy Arrays

Slicing a 1D Array

```
print(arr[1:4]) # Output: [20 30 40]
```

```
[20 30 40]
```

This slices the array to get a subarray from index 1 to 3.
Slicing is useful for extracting parts of an array.

Creating Numpy Arrays

Indexing a 2D Array

```
▶ arr = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])  
print(arr[0, 2]) # Output: 3
```

↔ 3

This accesses the element in the first row and third column of the array. 2D indexing requires specifying both row and column indices.

Creating Numpy Arrays

Slicing a 2D Array

```
▶ print(arr[1:, 1:]) # Output: [[5 6]  
# [8 9]]
```

```
↔ [[5 6]  
[8 9]]
```

This slices the 2D array to get elements from the second row onwards and the second column onwards. Slicing is essential for subsetting arrays.

Creating Numpy Arrays

Element-wise Addition

```
▶ arr1 = np.array([1, 2, 3])  
arr2 = np.array([4, 5, 6])  
print(arr1 + arr2) # Output: [5 7 9]
```

⇒ [5 7 9]

This performs element-wise addition of two arrays. Numpy automatically adds corresponding elements, making operations concise and efficient.

Creating Numpy Arrays

Array Broadcasting

```
arr = np.array([[1, 2, 3], [4, 5, 6]])  
print(arr + 10) # Output: [[11 12 13]  
                        #      [14 15 16]]
```

```
[[11 12 13]  
 [14 15 16]]
```

Broadcasting allows arithmetic operations on arrays of different shapes. Here, 10 is added to each element of the array.

- If a mathematical operation fails due to shape incompatibility or when broadcasting rules don't apply, Numpy will raise an error. This happens because Numpy can't carry out the operation as there is an unresolved dimensional mismatch.

Creating Numpy Arrays

Matrix Multiplication

```
▶ arr1 = np.array([[1, 2], [3, 4]])  
arr2 = np.array([[5, 6], [7, 8]])  
print(np.dot(arr1, arr2)) # Output: [[19 22]  
                             #         [43 50]]
```

⇒ $\begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$

This performs matrix multiplication using `np.dot()`, which calculates the dot product of two arrays. Useful for linear algebra applications.

Creating Numpy Arrays

Calculating Mean and Sum

```
▶ arr = np.array([1, 2, 3, 4, 5])  
print(np.mean(arr)) # Output: 3.0  
print(np.sum(arr))  # Output: 15
```

```
↔ 3.0  
15
```

These operations calculate the mean (average) and sum of the array elements. Numpy provides efficient built-in functions for statistical calculations.

Creating Numpy Arrays

Finding Maximum and Minimum Values

```
print(np.max(arr)) # Output: 5  
print(np.min(arr)) # Output: 1
```

```
5  
1
```

`np.max()` finds the maximum value, and `np.min()` finds the minimum value in the array. Useful for identifying the range of data.

Practical Applications

Square Matrix

A matrix with the same number of rows and columns.

```
import numpy as np

square_matrix = np.array([[1, 2, 3],
                           [4, 5, 6],
                           [7, 8, 9]])

print("Square Matrix:\n", square_matrix)
```

```
⇒ Square Matrix:
[[1 2 3]
 [4 5 6]
 [7 8 9]]
```

This is a 3x3 matrix where the number of rows equals the number of columns.

Diagonal Matrix

A matrix in which all off-diagonal elements are zero

```
[40] diagonal_matrix = np.diag([1, 2, 3])  
     print("Diagonal Matrix:\n", diagonal_matrix)
```

```
⇒ Diagonal Matrix:  
   [[1 0 0]  
    [0 2 0]  
    [0 0 3]]
```

The np.diag() function creates a diagonal matrix where only the elements along the main diagonal are non-zero.

Identity Matrix

A square matrix with ones on the main diagonal and zeros elsewhere.

```
identity_matrix = np.eye(3)  
print("Identity Matrix:\n", identity_matrix)
```

```
⇒ Identity Matrix:  
[[1. 0. 0.]  
 [0. 1. 0.]  
 [0. 0. 1.]]
```

The `np.eye(3)` function creates a 3x3 identity matrix. Identity matrices are used as the multiplicative identity in matrix algebra.

Upper Triangular Matrix

A matrix in which all the elements below the main diagonal are zero.

```
upper_triangular = np.array([[1, 2, 3],  
                             [0, 5, 6],  
                             [0, 0, 9]])  
  
print("Upper Triangular Matrix:\n", upper_triangular)
```

```
⇒ Upper Triangular Matrix:  
[[1 2 3]  
 [0 5 6]  
 [0 0 9]]
```

This matrix has zeros below the main diagonal. Upper triangular matrices are common in solving systems of linear equations.

Lower Triangular Matrix

A matrix in which all the elements above the main diagonal are zero.

```
▶ lower_triangular = np.array([[1, 0, 0],  
                               [4, 5, 0],  
                               [7, 8, 9]])  
  
print("Lower Triangular Matrix:\n", lower_triangular)
```

```
↔ Lower Triangular Matrix:  
[[1 0 0]  
 [4 5 0]  
 [7 8 9]]
```

This matrix has zeros above the main diagonal. Lower triangular matrices are used in various algorithms, such as Cholesky decomposition.

Symmetric Matrix

A matrix that is equal to its transpose, meaning $A = A^T$.

```
symmetric_matrix = np.array([[1, 2, 3],  
                             [2, 5, 6],  
                             [3, 6, 9]])  
print("Symmetric Matrix:\n", symmetric_matrix)
```

```
⇒ Symmetric Matrix:  
[[1 2 3]  
 [2 5 6]  
 [3 6 9]]
```

The matrix is symmetric because it remains the same when transposed. Symmetric matrices frequently appear in optimization problems.

Transpose of a Matrix

Flipping a matrix over its diagonal, switching the row and column indices.

```
matrix = np.array([[1, 2, 3],  
                  [4, 5, 6]])  
transpose_matrix = np.transpose(matrix)  
print("Original Matrix:\n", matrix)  
print("Transpose of Matrix:\n", transpose_matrix)
```

```
Original Matrix:  
[[1 2 3]  
 [4 5 6]]  
Transpose of Matrix:  
[[1 4]  
 [2 5]  
 [3 6]]
```

The `np.transpose()` function is used to find the transpose of a matrix, which switches the rows and columns.

Trace of a Matrix

The sum of all the elements on the main diagonal of a square matrix.

```
matrix = np.array([[1, 2, 3],  
                  [4, 5, 6],  
                  [7, 8, 9]])  
trace = np.trace(matrix)  
print("Trace of Matrix:", trace)
```

Trace of Matrix: 15

The `np.trace()` function calculates the trace of a matrix, which is often used in machine learning algorithms and various mathematical computations.

Determinant of a Matrix

A scalar value that is a function of a square matrix, often used in solving linear equations, calculating matrix inverses, and more.

```
▶ matrix = np.array([[4, 6],  
                     [3, 8]])  
determinant = np.linalg.det(matrix)  
print("Determinant of Matrix:", determinant)
```

↔ Determinant of Matrix: 14.000000000000004

The `np.linalg.det()` function computes the determinant, which provides important information about the matrix properties, such as invertibility.

Inverse of a Matrix

A matrix that, when multiplied with the original matrix, yields the identity matrix.

```
matrix = np.array([[1, 2],  
                  [3, 4]])  
inverse_matrix = np.linalg.inv(matrix)  
print("Inverse of Matrix:\n", inverse_matrix)
```

```
⇒ Inverse of Matrix:  
[[-2.  1.]  
 [ 1.5 -0.5]]
```

The `np.linalg.inv()` function finds the inverse of a matrix, which is useful for solving systems of linear equations.

Rank of a Matrix

The rank of a matrix is the maximum number of linearly independent column vectors in the matrix.

```
▶ matrix = np.array([[1, 2, 3],  
                    [4, 5, 6],  
                    [7, 8, 9]])  
  
rank = np.linalg.matrix_rank(matrix)  
print("Rank of Matrix:", rank)
```

↔ Rank of Matrix: 2

The `np.linalg.matrix_rank()` function calculates the rank, which indicates the dimension of the vector space spanned by the columns of the matrix.